Tutorial on Molecular Docking of Protein-DNA and Protein-RNA Using MolAICal and LightDock

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io or https://molaical.gitlab.io
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

1. Introduction

MolAICal can use the open-source LightDock[1] software (https://lightdock.org) and further enhances the docking accuracy of biomacromolecules through MM/GBSA or MM/PBSA methods. LightDock is an open-source (GPL-3.0 license), **docking framework** for **protein-protein**, **protein-peptide**, **protein-RNA,** and **protein-DNA** complexes. It handles flexible and challenging cases (e.g., transient interactions, low-affinity complexes, or systems requiring backbone/side-chain flexibility) **particularly well and serves as an extensible platform for testing new scoring functions, restraints**, or **optimization strategies.** LightDock adapts the Glowworm Swarm Optimization (GSO) algorithm[2], a bio-inspired swarm-intelligence method originally developed for multimodal function optimization.

This tutorial uses the small hyperthermophilic chromosomal protein Sac7d (from the archaeon Sulfolobus acidocaldarius), a double-stranded DNA duplex, as demonstration examples (PDB ID: 1AZP), and an RNA chain from PDB ID: 1A1T. Users can also refer to the tutorial of LightDock from https://lightdock.org/tutorials or https://lightdock.org/tutorials/0.9.3/index.html.

This tutorial is only applicable for completion on the Linux operating system! Only some software can be completed on Windows. Users can transfer files between Linux and Windows via SSH shell or other tools to better observe the running results.

2. Materials

2.1. Software requirement

1) MolAICal: https://molaical.github.io

2) NAMD (**CPU version**): https://www.ks.uiuc.edu/Research/namd/

(**Notice:** This tutorial can be run by **NAMD 2.x, 3.x, or a higher version.** For example, the command "**namd3**" substitutes for the command "**namd2**" in this tutorial **if you use NAMD 3.x version**. For a higher version of NAMD, users can use a similar replacement way as the previous example.)

3) PyMol: https://github.com/cgohlke/pymol-open-source-wheels , https://github.com/maabuu/pymol-wheels , https://pypi.org/project/pymol-open-source-whl/ , https://github.com/cnpem/PyMOL4Win

4) VMD: https://www.ks.uiuc.edu/Research/vmd

2.2. Example files

1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/026-protein_pn_docking

3. Procedure

3.1. Preparing for molecules

To address the library dependency issues in Linux, **the Linux version of MolAICal (Not for Windows version)** adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

To prevent the loss of installed programs (e.g., VMD and NAMD) when rebuilding the MolAICal image, refer to step 3 in **Appendix 1**.

Assuming users have configured the internal commands in MolAICal for VMD and NAMD using the following instructions:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD. Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

Open the tutorial material folder "026-protein_pn_docking":
#> cd 026-protein_pn_docking

✧ Load PDB file "1AZP.pdb" into PyMol, delete the hydrogens (option) and water of the complex firstly, and then use the steps of **Figure 1** to save "protein_A.pdb (Chain A)", and "DNA_BC.pdb (Chain B, C)", respectively.

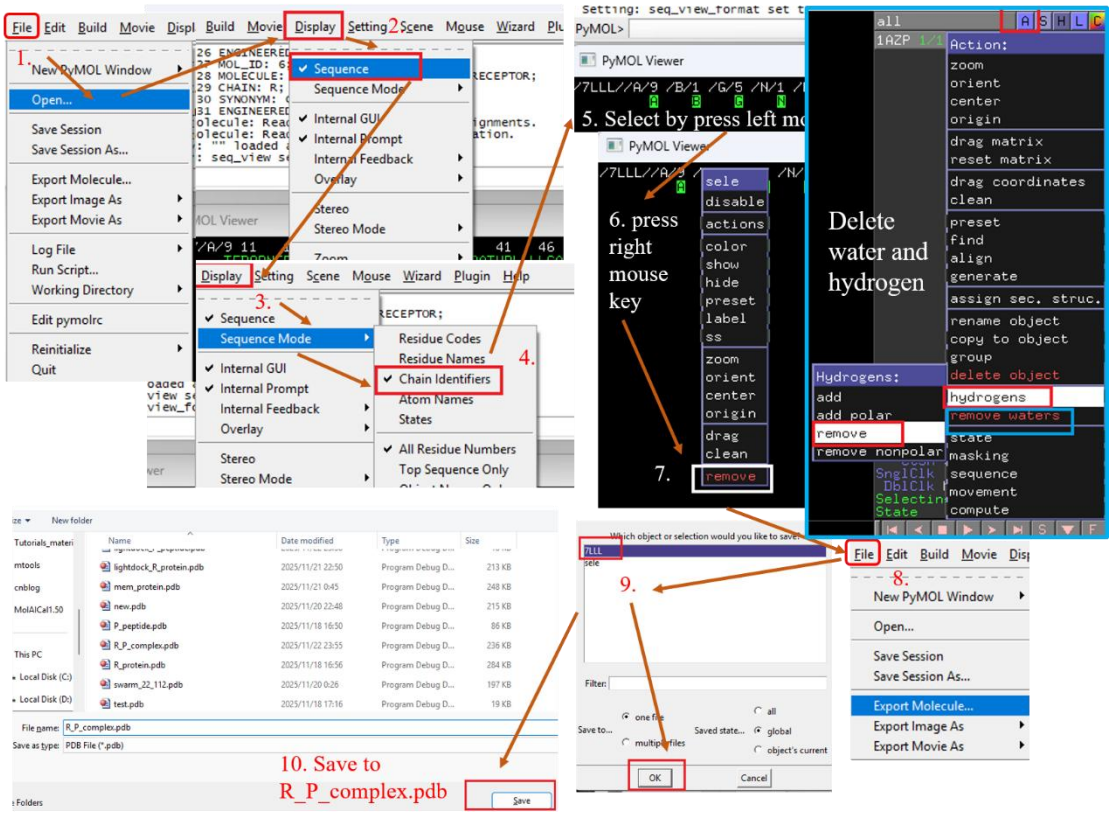

Figure 1

Here, "DNA_BC.pdb" contains two different chains, B and C, which do not meet the docking requirements for two molecules, so the two chains of "DNA_BC.pdb" need to be merged into a single chain B. The command is as follows:

```
#> molaical.exe -call run -c vmdargs -i -pdb DNA_BC.pdb -args none set sel [atomselect top all], \\\$sel set chain B, \\\$sel writepdb DNA_B.pdb
```

Note:

- It includes **all command-line arguments** of **external programs**, but the **'-i'** parameter must be the last one specified, while all other identifiers (e.g., '-call', '-c', etc.) must be **before '-i'.**
- **'-pdb'** means to load pdb file into VMD.
- **-c:** It will run the VMD command once in the Tk Console if its value is "vmdargs".
- The Strings after **'-args'** are represented to the commands which can be run on the Tk console of VMD.
  - The first parameter after **'-args'** can be a package name of VMD or **'none'**. If the first parameter is **'none'** after **'-args'**, it will omit the package for loading. For example, it will run the command "package require **autopsf**" in the Tk console of VMD if the first parameter is **'autopsf'** after **'-args'**. It will not run the command "package" if the first parameter is **'none'** after **'-args'**.
  - The string following **'-args'** contains one command that can be executed in VMD's Tk console before each comma **','**; in other words, the comma character replaces the process of entering each command in VMD's Tk console and pressing the Enter key. This allows multiple lines of commands to be executed with just a single command.
  - Some special characters, such as the character **"$"**, require the use of the escape character **"\"**. For special characters, MolAICal uses three escape characters **"\"** (i.e., **"\\\"**).

✧ In the same way above, load PDB file "1A1T.pdb" into PyMol, delete the hydrogens (option) and water of the complex firstly, and then use the steps of **Figure 1** to save " RNA_B.pdb (Chain B)".

3.2. Make residue restraint file

Creating a restraint file for residues is like identifying the key residue sites in the active pocket, around which the molecule will be docked. **Users can customize these key residues based on literature introductions or directly from experience**.

The residue representation format in the restraint file is:

```
Chain.Residue_Name.Residue_Number[.Residue_Insertion]
```

Note: Here, Residue_Insertion is an optional single-character identifier appended to the standard residue number, commonly used in PDB files to distinguish inserted residues without renumbering the entire sequence. In bioinformatics and structural biology, insertion codes are critical for accurately tracking and describing specific protein residues, particularly when dealing with sequence variations or comparisons.

When Residue_Insertion is omitted, the general format is:

```
Chain.Residue_Name.Residue_Number
```

Or

```
Chain.Residue_Name.Residue_Number P
```

Note: The label "P" indicates that this restraint is passive (as opposed to active restraints, which have no label). **Passive residue restraints are not recommended unless their implications are fully understood.**

Here, MolAICal is used to generate the residues list file (named 'rec.dat') of the receptor (protein) within 4.0 Å of the ligand (peptide) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args R P 4.0 rec.dat R
```

Note: the terms 1st, 2nd, 3rd, and 4th below represent the first argument, the second argument, the third argument, and the fourth argument, respectively, and so on.

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the scripts of MolAICal if its value is "sfile", here it calls the script file 'get_res.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** The chain name of the selected molecule;
- **2nd:** The chain name of reference molecule used for selection;
- **3rd:** The residues of the selected molecule within the specified distance from the reference molecule will be selected;
- **4th:** The saved filename;

- **5th:** Specifies the molecular type, 'R' stands for Receptor and 'L' stands for Ligand.

Similarly, MolAICal is used to generate the residues list file (named 'lig.dat') of the ligand (peptide) within 4.0 Å of the receptor (protein) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args P R 4.0 lig.dat L
```

Merge the 'rec.dat' and 'lig.dat' into the restraint file named 'restraints.list' as follows:

```
#> molaical.exe -call run -c ecommand -i cat rec.dat lig.dat > restraints.list
```

In many cases, the specific details of the ligand are unknown. **In this tutorial, the above commands are not used and are just supplied for more choices. This tutorial** only uses the key residues of the receptor (the first molecule), which are manually added the residues based on experience and the literature report for the constraint file named "restraints.list":

```
R A.TRP.24
R A.VAL.26
R A.ARG.42
```

3.3. Preparation for protein and nucleic acid

The protein partner must first possess hydrogen atoms parameterized in the DNA scoring function (based on the AMBER94 force field). If the below command does not work, please replace the below commands with the operations (e.g., delete and add hydrogens) of UCSF Chimera (https://www.cgl.ucsf.edu/chimera), which deal with molecules based on the AMBER force field.

3.3.1. For protein

Run the commands below:

Delete hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -Trim protein_A.pdb > protein_A_noH.pdb
```

Add hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -BUILD protein_A_noH.pdb > protein_A_H.pdb
```

Renumbered the atoms:

```
#> molaical.exe -call run -c ldock -i pdb_reatom protein_A_H.pdb > protein.pdb
```

3.3.2. For DNA

Run the commands below:

Delete hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -Trim DNA_B.pdb > DNA_B_noH.pdb
```

For adding hydrogen, there is a "resname" issue in the nucleic acids (DNA or RNA): The "reduce" command appears to be sensitive to the position of the resname, requiring that within the allowed resname range for nucleic acids, the name must be right-aligned. This issue is from the chain setting and save operations of VMD above.

Fix resname of DNA:

```
#> molaical.exe -call run -c sfile -i fix_pdb.tcl -args DNA_B_noH.pdb DNA_B_nh.pdb
```

Add hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -BUILD DNA_B_nh.pdb > DNA_B_H.pdb
```

Update the H types. The code in "reduce_to_amber.py" is: " *atom.name = translation[atom.name]* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i reduce_to_amber.py DNA_B_H.pdb DNA_dh.pdb
```

Delete the hydrogens "HO'3" and "HO'5" which conflict with the AMBER94 force field. The code in "purge_prime_H.py" is: " *if atom.name in ("HO'3", "HO'5"): continue* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i purge_prime_H.py DNA_dh.pdb DNA.pdb
```

3.3.3. For RNA

Run the commands below:

Delete hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -Trim RNA_B.pdb > RNA_nh.pdb
```

Add hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -BUILD RNA_nh.pdb > RNA_h.pdb
```

For the further preparation of RNA molecules, the following three requirements still need to be met:

- Hydrogen atoms added by "reduce" for nucleic acids are not fully name-compatible with the AMBER94 force field. (**reduce_to_amber.py:** the script to rename and/or remove incompatible atom types).
- Moreover, Reduce appends terminal hydrogens (at the 3' and/or 5' ends) to RNA, which leads to conflicts with the AMBER94 force field. (**purge_prime_H.py:** the script to remove the terminal (3' and/or 5') hydrogens).
- Finally, the AMBER94 force field requires RNA residues to be labeled with an "R" prefix. (**retag_rna.py:** One script to add or remove the "R" prefix for RNA residues)

1) Add "R" before the original resname of RNA. The code in "retag_rna.py" is " *atom.residue_name = "R" + atom.residue_name* "

```
#> molaical.exe -call run -c sfile -i retag_rna.py RNA_h.pdb rna_pre.pdb
```

2) Update the H types. The code in "reduce_to_amber.py" is: " *atom.name = translation[atom.name]* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i reduce_to_amber.py rna_pre.pdb rna_pre2.pdb
```

3) Delete the hydrogens "HO'3" and "HO'5" which conflict with the AMBER94 force field. The code in "purge_prime_H.py" is: " *if atom.name in ("HO'3", "HO'5"): continue* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i purge_prime_H.py rna_pre2.pdb rna_tagged.pdb
```

4) The **"-r"** argument of "retag_rna.py" means to remove the added tag "R" for the setup.

```
#> molaical.exe -call run -c sfile -i retag_rna.py -r rna_tagged.pdb RNA.pdb
```

3.3.4. Make the workdirs for DNA and RNA, respectively

In order to avoid the file conflict between DNA and RNA projects, they should move to different directories as the following commands:

```
# 1. Build the workdir for DNA and RNA by the command below:
#> molaical.exe -call run -c ecommand -i mkdir DNA RNA
```

```
# 2. Move protein and DNA file to the folder DNA by the commands below:
#> molaical.exe -call run -c ecommand -i cp -f protein.pdb DNA
#> molaical.exe -call run -c ecommand -i cp -f DNA.pdb DNA
```

```
# 3. Move protein and RNA file to the folder RNA by the commands below:
#> molaical.exe -call run -c ecommand -i cp -f protein.pdb RNA
#> molaical.exe -call run -c ecommand -i cp -f RNA.pdb RNA
```

3.4. Setup and Simulation for Molecular Docking

1) For DNA, open the folder DNA, and run setup and simulation:

```
#> cd DNA
#> molaical.exe -call run -c ldock -i lightdock3_setup.py protein.pdb DNA.pdb --noxt -sp -rst ../restraints.list
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s dna -c 8
```

2) For RNA, open the folder RNA, and run setup and simulation:

```
#> cd ../RNA
#> molaical.exe -call run -c ldock -i lightdock3_setup.py protein.pdb RNA.pdb --noxt -sp -rst ../restraints.list
## Simulation needs AMBER94 force field, which requires adding the R tag before "resname" of RNA.
#> molaical.exe -call run -c ecommand -i mv lightdock_RNA.pdb lightdock_rna_untagged.pdb
#> molaical.exe -call run -c sfile -i retag_rna.py lightdock_rna_untagged.pdb lightdock_RNA.pdb
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s dna -c 8
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:**
  - ✧ It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "ldock";
  - ✧ It will run the DOS command in Windows or run the shell bash command in Linux if its value is "ecommand";
  - ✧ It will run the VMD scripts of MolAICal if its value is "sfile".
- The 1st and 2nd arguments after "lightdock3_setup.py" are receptor and ligand files, respectively.
- **--noxt**: If this option is enabled, LightDock ignores OXT atoms. This is useful for several scoring functions that don't understand this special type of atom.
- **--anm:** If this option is enabled, the ANM mode is activated and backbone flexibility is modeled using ANM (via ProDy). **This parameter may cause structural distortion** in docking results by activating backbone flexibility modeled via ANM (using ProDy). **Avoid using it unless implementing better conformational sampling (e.g., energy minimization)**. Instead, pre-generate multiple conformations to bypass backbone flexibility during docking.
- **-sp:** If enabled (False by default), writes in the init folder debugging extra files.
- **-r, -rst restraints_file:** If restraints_file is provided, residue restraints will be considered during the setup and the simulation. If restraints_file is not provided, the setup and the simulation will focus on the entire receptor.
- The 1st and 2nd arguments after "lightdock3.py" are the configuration file generated on the setup step, and the number of steps of the simulation (here, it is 100 steps), respectively.
- **-s SCORING_FUNCTION:** The default scoring function (DFIRE, fast C implementation fastdfire) using this flag. A name of a scoring function or a file containing the name and weight of multiple scoring functions are accepted.

◆ **-c CORES:** By default, the total number of available CPU cores on the hardware is used to run the simulation, but a different number of CPU cores can be specified via this option.

✧ It will produce the folder named **'init',** which contains the molecules in PDB format that represent the glowworms of swarms for further docking.

✧ The new generated files are: 'lightdock_protein.pdb', 'lightdock_DNA.pdb', and 'lightdock_RNA.pdb'

Open 'lightdock_protein.pdb' and all PDB files with the prefix **"starting_positions*"** in the folder named **'init'** by PyMol in a similar way to Figure 2:

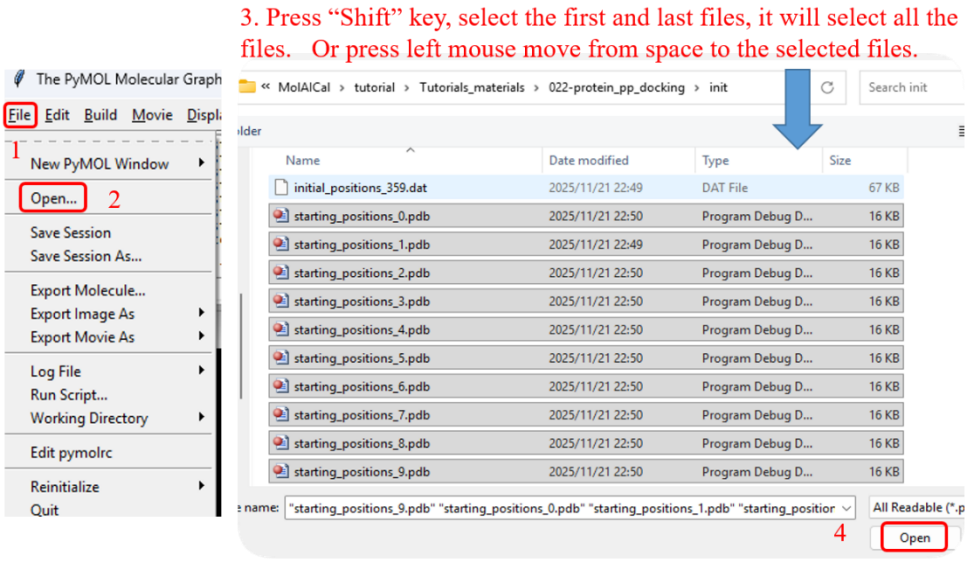

Figure 2

Please see the results in Figure 3. It is evident that many glowworms are **located around protein**. The file "restraints.list" can limit the number of glowworms; without this restraint file, more glowworms will be generated.

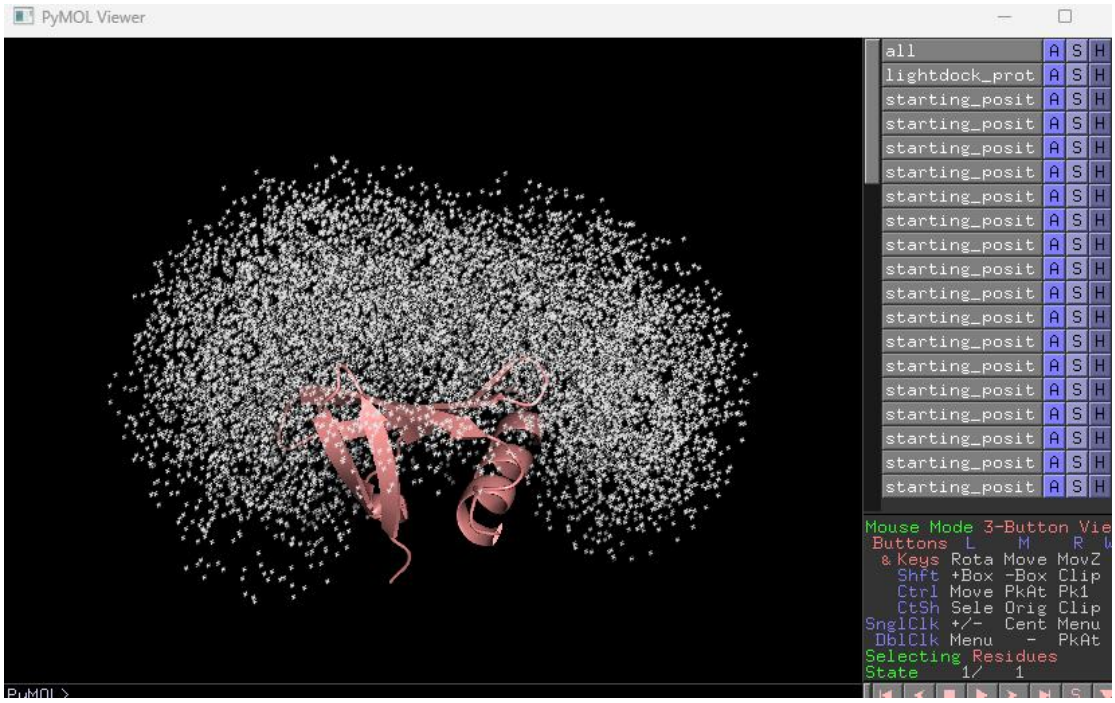

Figure 3

3.5. Generate models, Clustering, Rank, and filter

After the simulation is completed, MolAICal calls the script to generate models, cluster, rank, and filter the results as follows:

1) For DNA

```
#> molaical.exe -call run -c sfile -i lrank.sh 1::=protein.pdb 2::=DNA.pdb 3::=A 4::=B 5::=8 6::=../restraints.list 13::=0.7 17::=--lnuc
```

2) For RNA

This step needs to remove the tags "R" of RNA.

```
#> molaical.exe -call run -c ecommand -i mv lightdock_RNA.pdb lightdock_RNA_tagged.pdb
#> molaical.exe -call run -c ecommand -i mv lightdock_rna_untagged.pdb lightdock_RNA.pdb
```

Note: It seems only the RNA simulation step and "reduce_to_amber.py" need "R" tags to adapt for the AMBER force field, so the untagged and tagged files can be independently prepared.

```
#> molaical.exe -call run -c sfile -i lrank.sh 1::=protein.pdb 2::=RNA.pdb 3::=A 4::=B 5::=8 6::=../restraints.list 13::=0.7 17::=--lnuc
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::= '**: Here, it is used to assign values to parameters in the specified order.
- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".

- **'1::='**: It is the path of the first input molecule (receptor). Required parameters.
- **'2::='**: It is the path of the second input molecule (ligand). Required parameters.
- **'3::='**: Chains on the receptor partner (1st molecule). The default value is **'R'**.
- **'4::='**: Chains on the ligand partner (2nd molecule). The default value is **'P'**.
- **'5::='**: The number of used CPU cores. The default value is **12**.
- **'6::='**: File including restraints. The default value is **'restraints.list'**.
- **'13::='**: Structures with at least this fraction of native contacts for the filter step. The default value is **0.4**. To identify ligands that bind to more key residues in the receptor's active site, the constraint ratio is set to 0.7 here.
- **'17::='**: More options for "lgd_filter_restraints.py". The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma characters ','. MolAICal will automatically convert each comma ',' to a space " ". Here, the parameter "--lnuc" represents nucleic acids.
- Other parameters use the default values. For more information, please see the MolAICal manual.

After the clustering and filtering processes are complete, a new directory named **"filtered"** is generated. This directory holds all predicted structures that meet the default 70% filtering criteria. Within it, a file named "rank_filtered.list" will be found, which ranks these structures based on the "dna" scores, where higher (more positive) scores indicate better rankings.

- **dna** score**:** Implementation of the pyDockDNA scoring function (no desolvation) and custom Van der Waals weight for protein-DNA docking. Implemented using the Python C-API. It is also applicable to RNA scoring.

Table 1

Nucleic acid type	Name	Score	Percentages
DNA	swarm_7_101.pdb	217.290	1.000
	swarm_13_71.pdb	175.002	1.000
	swarm_48_142.pdb	172.741	1.000
	……..	……..	……..
RNA	swarm_18_31.pdb	237.259	1.000
	swarm_46_142.pdb	218.034	1.000
	swarm_14_41.pdb	213.206	1.000
	……..	……..	……..

Note: Percentages corresponding to >13::=0.7 is calculated by len(contacts_receptor & restraints_receptor) + len(contacts_ligand & restraints_ligand) / (len(restraints_receptor) + len(restraints_ligand)). The molecular naming format: swarm_{id_swarm}_{id_glowworm}.pdb, where 'id_swarm' corresponds to the swarm folder name generated during 'setup', and 'id_glowworm' refers to the pdb-formatted molecule name within the swarm folder.

Table 1 only shows the scores of the part docked results. Load the "1AZP.pdb (DNA complex)" or "1A1T.pdb (RNA complex)", which is the native binding conformation and the molecules in Table 1 to **PyMol**, align the "1AZP.pdb (DNA complex)" or "1A1T.pdb (RNA complex)" to the wanted molecule (see Figure 4). After alignment, it can further examine the docking conformations and other results.

Besides, users can put the best one or the assigned top molecules to a new folder

"**filtered**/candidates" as command below:

```
#> molaical.exe -call run -c sfile -i mrank.py -ft 1
```

➢ -t: Number of top molecules to copy to candidates (default: 0)

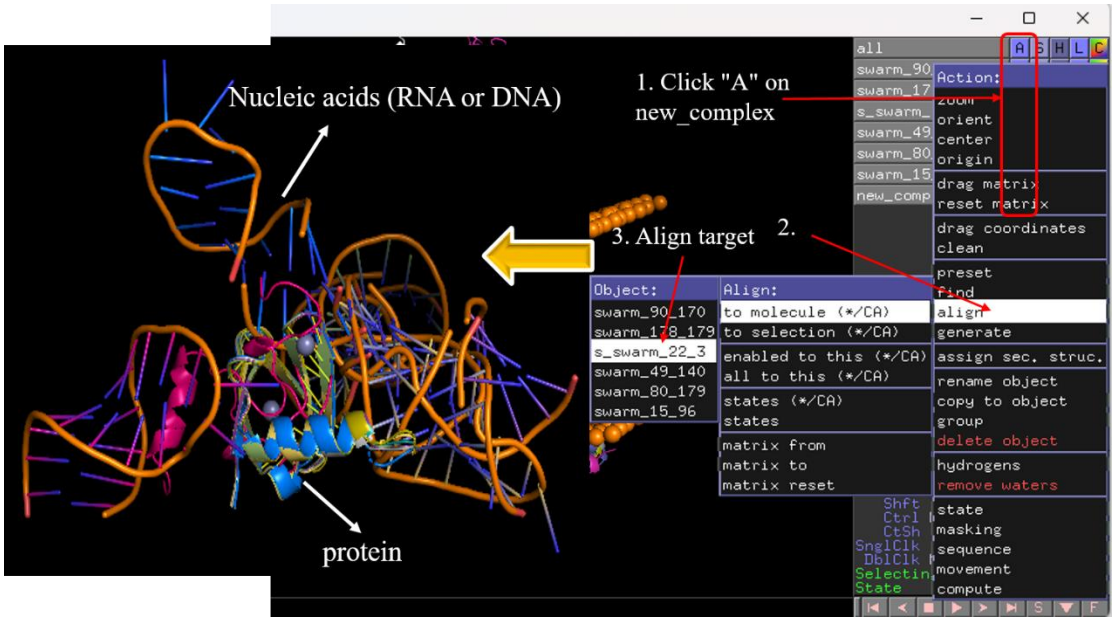

Figure 4

3.6. MM/GBSA calculation for further optimization

MolAICal computes MM/GBSA and MM/PBSA binding free energies. Based on literature evidence and empirical validation, MM/GBSA demonstrates superior performance over MM/PBSA for protein-protein and **protein-nucleic acids** docking systems. However, this does not imply that MM/GBSA universally outperforms MM/PBSA across all systems. We recommend that researchers select the optimal method based on their specific biomolecular system and computational objectives. Here, the workflow employs MM/GBSA to calculate binding affinities for the docking poses as the command below:

1) For DNA

```
#> molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=A 2::=B
```

2) For RNA

```
#> molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=A 2::=B
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::= '**: Here, it is used to assign values to parameters in the specified order.

- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".
- **'1::='**: Chain of the first molecule. The default value is 'R'.
- **'2::='**: Chain of the second molecule. The default value is 'P'.
- **'3::='**: The number of used CPU cores. The default value is **12**.
- Other parameters use the default values. For more information, please see the MolAICal manual.

Notice:

1) During initial execution, a parameter file named **"cal_mmgbpbsa.sh"** is automatically generated. Subsequent runs will not overwrite an existing **"cal_mmgbpbsa.sh"** file; the current version in the directory takes precedence. Users may customize parameters within **"cal_mmgbpbsa.sh"**, which **adheres** to **MolAICal's script format** enabling **batch execution** of MolAICal commands line-by-line (command-line calls typically begin with molaical.exe).

2) MolAICal currently only supports automatic calculation of MM/GBSA and MM/PBSA for standard protein residues and nucleic acids. For small molecules and non-standard amino acids, additional CHARMM force fields are required, and the script named "cal_mmgbpbsa.sh" in the directory can be modified by adding parameters such as "-top" and "-ff" to extend the additional CHARMM force fields. Specifically, changing the 5th line of "cal_mmgbpbsa.sh" switches the calculation to MM/PBSA. Detailed information can be found in the MolAICal manual and corresponding MM/GBSA and MM/PBSA tutorials.

Ranking or extracting the wanted molecules to the './mmgbpbsa /candidates' as the commands below:

1. Only print the rank results of MM/GB/PBSA, run command below:

```
#>molaical.exe -call run -c sfile -i mrank.py
```

2. Copy the top 2 molecules of the results to the **'./mmgbpbsa /candidates', run command below:**

```
#> molaical.exe -call run -c sfile -i mrank.py -t 2
```

➢ -t: Number of top molecules to copy to candidates (default: 0)

It will show the results as below:

1) For DNA

Name	G_binding (kcal/mol)	±	SE	SD
swarm_7_101.pdb	-33.6310	+/-	0.0389	0.2750
swarm_49_130.pdb	-18.4513	+/-	0.0032	0.0225
swarm_13_71.pdb	-17.9355	+/-	0.0163	0.1154
swarm_22_101.pdb	-10.2513	+/-	0.0042	0.0297
swarm_33_151.pdb	-5.6938	+/-	0.0115	0.0814
swarm_48_142.pdb	-1.5483	+/-	0.0135	0.0952

2) For RNA

Name	G_binding (kcal/mol)	±	SE	SD
swarm_14_41.pdb	-25.2272	+/-	0.0159	0.1123
swarm_18_31.pdb	-22.5430	+/-	0.0067	0.0471
swarm_14_142.pdb	-19.1931	+/-	0.0147	0.1041
swarm_46_142.pdb	-10.4946	+/-	0.0197	0.1395
swarm_18_83.pdb	-8.8363	+/-	0.0149	0.1052
swarm_33_158.pdb	-6.7762	+/-	0.0346	0.2444

Under normal circumstances, the more negative the G_binding value, the better.

Notice: This tutorial provides detailed, **multi-step instructions primarily for teaching purposes**. While the **steps** in this tutorial have been **updated**, the tutorial **results** have **not** been **refreshed**; if your output differs from the tutorial's, trust your actual results.

If the program is terminated abnormally (**e.g., via "Ctrl+C" or other non-standard methods**), **numerous residual processes may continue running uncontrollably.** To resolve this issue, users can delete all processes associated with the current directory using the following command::

```
#> molaical.exe -eset pdclean
```

Note:

- Execute this command multiple times to ensure complete removal of related processes.
- WARNING: This will also terminate processes of OTHER RUNNING PROGRAMS in the same directory.
- Verify carefully before execution:
  - If multiple programs are running from this directory, explicitly confirm which directory-associated processes are safe to delete.
  - Never run this command in shared/production directories without explicit validation."

References

1. Jiménez-García, B.; Roel-Touris, J.; Romero-Durana, M.; Vidal, M.; Jiménez-González, D.; Fernández-Recio, J., LightDock: a new multi-scale approach to protein-protein docking. *Bioinformatics* **2018,** *34* (1), 49-55.
2. Krishnanand, K. N.; Ghose, D., Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions. *Swarm Intelligence* **2009,** *3* (2), 87-124.

Appendix 1

Install external programs inside the MolAICal container (if finished, only for Linux Version MolAICal)

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To address the library dependency issues in Linux, the Linux version of MolAICal adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter

appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may be due to permission issues with the mounted file system, which can be ignored.

◆ Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

◆ Now, change the loaded new container name to the default container name "molaical", the

previous container is renamed to "bakmolaical", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#>molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container** (**dynamic**) is generated from an **image** (**static**). Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.

For more information, please see the manual of MolAICal or run "molaical.exe -eset sys --help" to get more help details.